
LLM Fundamentals

Transformer Architecture · Tokenization & Context · Embeddings & Similarity

Chapter 1 Transformer Basics — Attention · Tokens · Embeddings
Overview

Chapter 2 Tokenization & Context — Token Limits · Context Windows ·
Cost

Chapter 3 Embeddings & Similarity — Vectors · Cosine Similarity ·
Semantic Search

Enterprise AI Training Series
Target Audience: Freshers & Associate Engineers

Table of Contents

Chapter 1 — Transformer Basics

- 1.1 What is a Transformer?
- 1.2 The Attention Mechanism
- 1.3 Multi-Head Attention
- 1.4 Tokens vs Words
- 1.5 Embeddings Overview
- 1.6 The Full Transformer Pipeline

Chapter 2 — Tokenization & Context Windows

- 2.1 What is Tokenization?
- 2.2 Common Tokenization Algorithms
- 2.3 Token Limits & Context Windows
- 2.4 Cost Implications of Tokens
- 2.5 Practical Strategies

Chapter 3 — Embeddings & Similarity

- 3.1 Vector Embeddings Deep Dive
- 3.2 How Embeddings Are Trained
- 3.3 Cosine Similarity
- 3.4 Semantic Search Basics
- 3.5 Embedding Models Comparison
- 3.6 End-to-End Python Example

Transformer Basics

Attention Mechanism · Tokens vs Words · Embeddings Overview

1.1 What is a Transformer?

The Transformer is a deep learning architecture introduced in the 2017 paper *Attention Is All You Need* by Vaswani et al. at Google. Before Transformers, NLP relied on RNNs and LSTMs that processed text sequentially — one word at a time. This made training slow and made it hard to capture long-range dependencies. Transformers solved this by processing all tokens **in parallel** using **Self-Attention**, enabling every token to relate to every other token simultaneously regardless of distance.

RNN vs Transformer — At a Glance

Feature	RNN / LSTM	Transformer
Processing Order	Sequential (word by word)	Parallel (all tokens at once)
Long-range Dependencies	Poor — vanishing gradients	Excellent — via attention
Training Speed	Slow	Fast (GPU parallelism)
Context Length	Limited (~100 tokens)	Large (thousands of tokens)
Scalability	Hard to scale	Scales to billions of parameters
Examples	LSTM, GRU, Bi-LSTM	GPT, BERT, T5, LLaMA, Mistral

1.2 The Attention Mechanism

Attention lets the model decide *which tokens to focus on* when encoding each token. When understanding a word, not all other words in the sentence are equally relevant — attention assigns a learned weight to each.

The Query–Key–Value (QKV) Framework

- **Query (Q)** — *What am I looking for?*

The current token's information need. Analogous to a search query sent to a database.

- **Key (K)** — *What do I have to offer?*

Each token's identifier. Analogous to the index keys in that database.

- **Value (V)** — *What is my actual content?*

The information payload returned when a key sufficiently matches the query.

Attention Score Formula

The scaled dot-product attention is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) \times V$$

$Q \times K^T$ – dot product: measures similarity between query and each key
 $/ \sqrt{d_k}$ – scaling factor (d_k = key dimension) to prevent gradient vanishing
 $\text{softmax}()$ – converts raw scores to probabilities that sum to 1.0
 $\times V$ – weighted sum of values using those attention probabilities

Worked Example: Disambiguating 'bank'

For the sentence "**The bank by the river was steep**", the word **bank** is ambiguous. The attention weights learned for the token "bank" are approximately:

Token Pair	Attention Weight	Interpretation
bank → river	0.62 (High)	Learns bank = riverbank from this context
bank → steep	0.18 (Medium)	Slope reinforces riverbank meaning
bank → the	0.08 (Low)	Articles carry little disambiguating info
bank → was	0.07 (Low)	Auxiliary verb — minimal signal
bank → bank	0.05 (Low)	Self-attention baseline

■ Key Insight

The model does not need hand-crafted rules. By training on massive corpora, attention weights are learned automatically to capture contextual relationships.

1.3 Multi-Head Attention

A single attention head learns one type of relationship. **Multi-Head Attention** runs multiple heads in parallel, each learning different linguistic patterns, then concatenates and linearly projects the results.

Head	What It Learns	Example Pattern
Head 1	Syntactic structure	Subject–verb agreement, noun–modifier pairing
Head 2	Coreference resolution	Linking pronouns to antecedents (she → Alice)
Head 3	Semantic similarity	Relating synonyms or conceptually close words
Head 4	Positional proximity	Attending to nearby tokens regardless of meaning
Head N	Task-specific patterns	Complex learned relationships for downstream tasks

1.4 Tokens vs Words

A common misconception: tokens ≠ words. Most LLMs use **subword tokenization** — common words are kept whole; rare or complex words are split into smaller subword units.

Input Text	Tokens (GPT-4 style)	Count
"Hello world"	['Hello', ' world']	2
"Unbelievable"	['Un', 'believ', 'able']	3

"ChatGPT"	['Chat', 'G', 'PT']	3
"I love NLP!"	['I', ' love', ' NLP', '!']	4
"antidisestablishmentarianism"	['ant','idis','estab','lish','ment','arian','ism']	7

■ ■ Rule of Thumb

In English: 1 token $\approx$ 0.75 words. 100 tokens $\approx$ 75 words. A full page of text $\approx$ 500–700 tokens. Code, non-English text, and technical jargon tend to use more tokens per word.

1.5 Embeddings Overview

Before a Transformer processes text, each token is converted into a dense numerical vector called an **embedding** (typically 512 – 4096 dimensions). Semantic relationships are encoded as geometric distances in this high-dimensional space.

Well-known properties of trained embedding spaces:

- king – man + woman $\approx$ queen (the classic Word2Vec analogy)
- Paris – France + Italy $\approx$ Rome (capital-city relationship preserved)
- Synonyms cluster close together; antonyms are more distant
- Semantically related words (dog, puppy, canine) form tight neighbourhoods

Embedding Type	Description	Used In
Token Embeddings	Each vocabulary token $\rightarrow$ fixed vector; learned at pre-training	Every LLM
Positional Embeddings	Encode position in sequence (1st, 2nd, 3rd...)	Every Transformer
Sentence Embeddings	Entire sentence/paragraph $\rightarrow$ single retrieval vector	RAG, Search, Clustering
Fine-tuned Embeddings	Adapted for a specific domain (legal, medical, finance)	Enterprise AI

1.6 The Full Transformer Input Pipeline

How the text **"I love transformers"** flows through the Transformer:

Step	Stage	Detail
1	Raw Text	"I love transformers"
2	Tokenisation	["I", "love", "transform", "ers"] $\rightarrow$ IDs: [40, 2005, 5276, 364]
3	Token Embeddings	Each ID $\rightarrow$ 768-dim vector e.g. [0.12, -0.45, 0.87, ...] $\times$ 768
4	Pos. Embeddings	Positional signal added: pos[0], pos[1], pos[2], pos[3]
5	Transformer Layers	$N \times$ (Multi-Head Attention + Feed-Forward + Layer Norm)
6	Output	Contextualised vectors $\rightarrow$ logits $\rightarrow$ next-token probability distribution

CHAPTER 2

Tokenization & Context Windows

Token Limits · Context Windows · Cost Implications

2.1 What is Tokenization?

Tokenization converts raw text into a sequence of integers called **token IDs**. The tokenizer uses a fixed **vocabulary** (typically 32,000 – 128,000 tokens) learned during pre-training. Every string the model sees — prompts, documents, and API responses — is tokenized before processing.

2.2 Common Tokenization Algorithms

Byte Pair Encoding (BPE)

Starts with individual characters, then repeatedly merges the most frequent adjacent pair. Used by GPT-2, GPT-3, GPT-4, and LLaMA 2/3.

Strength

Efficient, language-agnostic, handles rare words gracefully

WordPiece

Similar to BPE but optimises the likelihood of training data rather than raw frequency. Used by BERT and Google's models.

Strength

Strong for masked language modelling tasks

SentencePiece

Treats input as a raw stream of Unicode characters — no pre-tokenization needed. Handles any language including CJK scripts. Used by T5 and LLaMA.

Strength

Excellent for multilingual models

Unigram Language Model

Starts with a large vocabulary and prunes tokens that minimally affect training likelihood. Often combined with SentencePiece.

Strength

Very flexible, good compression ratio

2.3 Token Limits & Context Windows

The **context window** is the maximum number of tokens a model can process in one API call — this includes both the prompt (input) and the completion (output). Exceeding the limit results in an error or silent truncation.

Context Window Reference — Major Models (2024–2025)

Model	Context Window	Input Cost / 1M tok	Notes
GPT-4o	128,000 tokens	\$5.00	OpenAI flagship
GPT-4o mini	128,000 tokens	\$0.15	Budget-friendly GPT-4 class
Claude 3.5 Sonnet	200,000 tokens	\$3.00	Anthropic; large context
Claude 3 Haiku	200,000 tokens	\$0.25	Fastest Anthropic model
Gemini 1.5 Pro	1,000,000 tokens	\$3.50	Google 1M-token 'long context'
Gemini 1.5 Flash	1,000,000 tokens	\$0.35	Cheaper Gemini long-context
LLaMA 3.1 405B	128,000 tokens	Self-host	Meta open-source
Mistral Large 2	128,000 tokens	\$3.00	Mistral AI

■ ■ Pricing Note

Pricing above is approximate and changes frequently. Always check the provider's official pricing page before budgeting a project.

What Counts Toward the Context Window?

- **System Prompt:** Instructions given to the model at the start of every session
- **Conversation History:** All previous user and assistant turns
- **Retrieved Documents:** RAG chunks fetched from a vector database
- **User Message:** The current user input
- **Model Output:** The generated completion (billed as output tokens)

2.4 Cost Implications of Tokens

Token costs are the primary driver of LLM API expenditure in production systems.

Billing Metric	Definition	Typical Price Range
Input Tokens	Tokens in your prompt + context	\$0.15 – \$30 per 1M tokens
Output Tokens	Tokens in the model's response	\$0.60 – \$120 per 1M tokens (3–4× input cost)
Cached Input	Repeated prompt prefix (some providers)	50–90% discount vs standard input
Batch API	Async, non-real-time processing	~50% discount (OpenAI, Anthropic)

Cost Calculation Example

Scenario: Customer support chatbot, 10,000 conversations/day, avg 800 input + 200 output tokens, using GPT-4o (\$5.00/\$15.00 per 1M):

Daily input tokens = $10,000 \times 800 = 8,000,000$ tokens
Daily output tokens = $10,000 \times 200 = 2,000,000$ tokens

Input cost = $8,000,000 / 1,000,000 \times \$5.00 = \$40.00$
Output cost = $2,000,000 / 1,000,000 \times \$15.00 = \$30.00$
Daily total = $\$70.00 \rightarrow$ Monthly $\approx \$2,100$

Switching to GPT-4o mini ($\$0.15 / \0.60 per 1M):
Daily total $\approx \$2.40 \rightarrow$ 97% cost reduction for the same workload

■ Cost Optimisation

Always prototype with smaller, cheaper models first. Reserve flagship models for tasks that genuinely require their capability.

2.5 Practical Strategies to Manage Token Usage

► Prompt Compression

Remove unnecessary words, whitespace, and redundant instructions from system prompts. A 2,000-token system prompt costs 60× more than a 33-token one across 100K calls.

► Chunking & RAG

Instead of stuffing entire documents into the context window, use Retrieval Augmented Generation: embed documents, retrieve only the top-k relevant chunks, inject those chunks. This keeps context lean and reduces hallucinations.

► Conversation Summarisation

For long multi-turn chats, periodically summarise earlier turns. Replace raw history with a compact summary to stay within token limits.

► Model Tier Selection

Use small/cheap models (Haiku, GPT-4o mini) for simple classification, routing, or extraction. Reserve powerful models for complex reasoning or generation.

► Output Length Control

Set `max_tokens` appropriately. If you only need a yes/no answer, cap output at 10 tokens. Unconstrained outputs are often 3–4× longer than necessary.

► Prompt Caching

Structure prompts so the static system prompt appears first. Repeated prefixes are cached and billed at a heavy discount on subsequent calls (Anthropic, OpenAI).

CHAPTER 3

Embeddings & Similarity

Vector Embeddings · Cosine Similarity · Semantic Search

3.1 Vector Embeddings — Deep Dive

An **embedding** is a dense numerical vector representing the meaning of a piece of text (token, sentence, paragraph, or document). Unlike one-hot vectors (sparse, no semantic meaning), embeddings are **dense**, **continuous**, and encode semantic similarity as geometric proximity in vector space.

Property	One-Hot Encoding	Dense Embedding
Dimensionality	Vocab size (50,000+)	Fixed small dim (128–4096)
Sparsity	Extremely sparse (1 non-zero entry)	Dense (all values non-zero)
Semantic meaning	None — arbitrary numeric IDs	Geometric closeness = meaning similarity
Memory	Very large	Compact and efficient
Similarity check	Impossible	Dot product or cosine distance

3.2 How Embeddings Are Trained

Word2Vec (2013)

Trains a shallow neural network on two tasks: (a) CBOW — predict a word from its context, or (b) Skip-Gram — predict context words from a central word. The hidden layer weights become the embeddings.

Key Strength

Word-level analogies; fast training

GloVe (2014)

Global Vectors — factorises a global word co-occurrence matrix. Captures both local and global corpus statistics.

Key Strength

Better global context than Word2Vec

BERT Contextual Embeddings (2018)

Produces different embeddings for the same word depending on context. "bank" in a financial sentence vs. a river sentence gets different vectors. Trained via Masked Language Modelling (MLM).

Key Strength

Contextually rich; gold standard for NLU tasks

Sentence Transformers (2019+)

Fine-tunes BERT-like models with Siamese networks and contrastive loss on sentence pairs. Optimised for producing a single fixed-size vector per sentence, ideal for retrieval and clustering.

Key Strength
Fast retrieval at scale; excellent semantic similarity

3.3 Cosine Similarity

Cosine similarity measures the **angle** between two vectors, not their magnitude. This makes it robust to differences in text length — a short sentence and its expanded version share similar meaning even if their raw vector magnitudes differ.

Formula

$$\text{cosine_similarity}(A, B) = (A \cdot B) / (||A|| \times ||B||)$$

$A \cdot B$ = sum of element-wise products (dot product)
 $||A||$ = $\text{sqrt}(\text{sum}(a_i^2))$ (Euclidean norm / magnitude)

Interpretation Scale

Score Range	Meaning	Example Pair
1.0	Identical direction — perfectly similar	"cat" vs "cat"
0.85 – 0.99	Highly similar — strong semantic match	"automobile" vs "car"
0.60 – 0.84	Moderately similar — related topics	"king" vs "queen"
0.30 – 0.59	Weakly related	"dog" vs "cloud"
~0.0	Orthogonal — unrelated	"cat" vs "justice"
-1.0 – 0.0	Opposite direction — antonyms / negation	"good" vs "terrible"

Python Example

```
import numpy as np

# Simplified 4-dim embeddings (real models use 768-4096 dims)
vec_cat = np.array([0.8, 0.6, 0.1, 0.0]) # 'cat'
vec_dog = np.array([0.7, 0.7, 0.1, 0.0]) # 'dog'
vec_sky = np.array([0.0, 0.1, 0.9, 0.8]) # 'sky'

def cosine_sim(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

print(cosine_sim(vec_cat, vec_dog)) # → 0.98 (very similar)
print(cosine_sim(vec_cat, vec_sky)) # → 0.11 (unrelated)
```

3.4 Semantic Search Basics

Traditional keyword search matches exact words. **Semantic search** uses embeddings to find results that are conceptually similar even without shared keywords. The query *'how do I fix a flat tyre?'* retrieves documents about *'puncture repair'* and *'changing a wheel'* even if those exact words never appear in the query.

End-to-End Semantic Search Pipeline

1. Corpus Ingestion

Load all documents (PDFs, web pages, knowledge base articles).

2. Chunking

Split documents into chunks of 256–512 tokens with ~50-token overlap to preserve context across chunk boundaries.

3. Embedding Generation

Call an embedding model API (or run locally) to convert each chunk into a vector.

4. Vector Store Indexing

Store vectors in a vector database (Pinecone, Qdrant, Weaviate, Chroma, pgvector). Build an ANN index (HNSW, IVF) for fast approximate nearest-neighbour lookup.

5. Query Embedding

At query time, embed the user's question using the same embedding model.

6. ANN Search

Find the top-k most similar chunks using cosine similarity in the vector store.

7. LLM Generation (RAG)

Inject retrieved chunks as context into the LLM prompt. The LLM synthesises an answer grounded in retrieved facts — reducing hallucination.

3.5 Embedding Models Comparison

Model	Dims	Max Tok	MTEB	Provider	Cost
text-embedding-3-large	3072	8,192	~64.6	OpenAI	\$0.13/1M
text-embedding-3-small	1536	8,192	~62.3	OpenAI	\$0.02/1M
text-embedding-ada-002	1536	8,192	~61.0	OpenAI	\$0.10/1M
voyage-large-2-instruct	1024	16,000	~68.3	Voyage AI	\$0.12/1M
cohere-embed-v3	1024	512	~64.5	Cohere	\$0.10/1M
all-MiniLM-L6-v2	384	256	~56.3	HuggingFace	Free OSS
bge-large-en-v1.5	1024	512	~63.9	BAAI	Free OSS
nomic-embed-text	768	8,192	~62.4	Nomic AI	Free OSS

■ ■ About MTEB

MTEB (Massive Text Embedding Benchmark) is the standard benchmark evaluating embedding models across retrieval, clustering, classification, and semantic similarity. Higher = better.

3.6 Full End-to-End Python Example

Complete semantic search using OpenAI embeddings and cosine similarity (no external vector DB — suitable for up to ~10K documents):

```
from openai import OpenAI
import numpy as np

client = OpenAI(api_key='YOUR_API_KEY')

# Step 1: Define corpus
corpus = [
    'How to fix a flat tyre on your car',
    'Guide to changing a bicycle wheel',
    'Best recipes for chocolate cake',
    'Puncture repair kit instructions',
    'Machine learning model evaluation metrics',
]

# Step 2: Embed all documents
def embed(texts):
    resp = client.embeddings.create(
        model='text-embedding-3-small', input=texts)
    return np.array([d.embedding for d in resp.data])

doc_embeddings = embed(corpus)

# Step 3: Embed the query
query = 'my tyre has a puncture, what should I do?'
q_emb = embed([query])[0]

# Step 4: Compute cosine similarities
def cosine_sim(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

scores = [cosine_sim(q_emb, d) for d in doc_embeddings]

# Step 5: Return top-2 results
ranked = sorted(enumerate(scores), key=lambda x: x[1], reverse=True)
for idx, score in ranked[:2]:
    print(f'Score: {score:.3f} | {corpus[idx]}')

# Output:
# Score: 0.921 | How to fix a flat tyre on your car
# Score: 0.879 | Puncture repair kit instructions
```

■ Why Semantic Search Works

The query 'my tyre has a puncture' matched 'flat tyre' and 'puncture repair' with high scores — with zero keyword overlap in the first result. This is the power of embeddings.

Chapter Summary — Key Concepts at a Glance

Concept	One-Line Summary
Transformer	Neural architecture that processes all tokens in parallel using attention
Self-Attention	Each token attends to all others to understand contextual meaning
QKV Mechanism	Query × Key → attention weights; weighted sum of Values → output
Multi-Head Attn.	Multiple parallel attention heads capture different linguistic patterns
Token	Subword unit processed by LLMs; ~0.75 words per token in English
BPE	Byte Pair Encoding — standard subword tokenizer used by GPT models
Context Window	Maximum tokens (input + output) in a single model API call
Token Cost	APIs charge per input and output token; output is usually 3–4× pricier
Embedding	Dense numerical vector encoding the semantic meaning of text
Cosine Similarity	Angle-based similarity: 1.0 = identical; 0 = unrelated; -1 = opposite
Semantic Search	Embed query + docs; retrieve top-k nearest neighbours by cosine sim
RAG	Retrieval Augmented Generation: inject retrieved chunks into LLM prompt
MTEB	Standard benchmark for comparing embedding model quality
Vector Database	Specialised store for embeddings with ANN search (Pinecone, Chroma...)

What's Next?

- **Prompt Engineering** — System prompts, few-shot examples, chain-of-thought, temperature control
- **RAG Architecture** — Full pipeline: chunking strategies, vector DB setup, reranking, evaluation
- **Fine-Tuning LLMs** — LoRA, QLoRA, instruction tuning, RLHF — adapting models to your domain
- **LLM Evaluation** — BLEU, ROUGE, RAGAS, human eval — measuring model output quality
- **LLMOps & Deployment** — LangChain, LlamaIndex, Guardrails, monitoring, cost tracking in prod
- **Multimodal Models** — Vision-language models (GPT-4V, LLaVA), audio models (Whisper)

■ Document Notice

This document is part of the Enterprise AI Training Series. Reproduction for commercial use without written permission is not permitted. For training delivery, lab guides, and workshop facilitation, contact your programme coordinator.